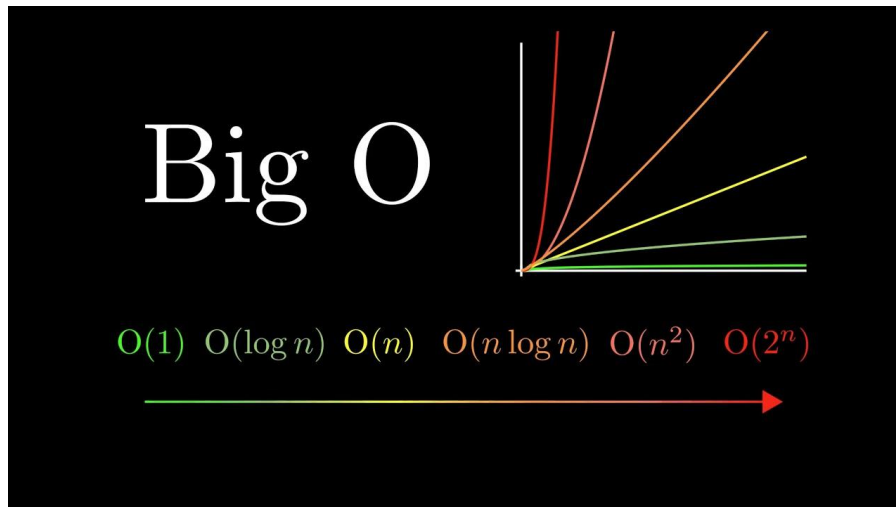


Justificación teórica de la complejidad temporal (Big O)



Búsqueda lineal – $O(n)$

La búsqueda lineal recorre la lista de transacciones elemento por elemento hasta encontrar el ID buscado o llegar al final de la colección. En el peor de los casos debe inspeccionar todos los elementos, por lo que su complejidad temporal es $O(n)$. A medida que aumenta la cantidad de transacciones, el tiempo de ejecución también crece de forma aproximadamente proporcional.

Búsqueda binaria – $O(\log n)$

La búsqueda binaria posee una complejidad temporal de $O(\log n)$, ya que en cada iteración reduce a la mitad el espacio de búsqueda. Para que este algoritmo funcione correctamente es indispensable que la lista se encuentre previamente ordenada por el criterio de búsqueda (ID de la transacción). Gracias a esta característica, la cantidad de comparaciones necesarias es considerablemente menor que en la búsqueda lineal, especialmente cuando el tamaño de la lista es grande.

Bubble Sort – $O(n^2)$

Bubble Sort utiliza dos recorridos anidados sobre la lista para comparar e intercambiar elementos hasta dejarlos ordenados. Debido a esta estrategia, en el peor caso realiza aproximadamente n^2 comparaciones, por lo que su complejidad temporal es $O(n^2)$. Esto provoca que el tiempo de ejecución aumente rápidamente cuando crece la cantidad de datos, convirtiéndose en un algoritmo poco eficiente para listas grandes.

TimSort (List.sort()) – $O(n \log n)$

El método List.sort() de Java utiliza el algoritmo TimSort, cuya complejidad promedio es $O(n \log n)$. Este algoritmo combina características de Merge Sort e

Insertion Sort, aprovechando secuencias que ya se encuentran parcialmente ordenadas para mejorar su rendimiento. Como consecuencia, resulta significativamente más eficiente que Bubble Sort para conjuntos de datos medianos y grandes.

Análisis de los resultados obtenidos

Las mediciones realizadas mediante `System.nanoTime()` muestran un comportamiento consistente con la teoría de complejidad algorítmica. La búsqueda binaria requiere muchas menos comparaciones que la búsqueda lineal, manteniendo tiempos de ejecución muy bajos incluso con grandes cantidades de transacciones. De manera similar, TimSort presenta un rendimiento ampliamente superior al de Bubble Sort, diferencia que se vuelve cada vez más evidente conforme aumenta el tamaño del conjunto de datos.

En conclusión, los resultados experimentales obtenidos durante el desarrollo del trabajo práctico coinciden con el análisis teórico de la notación Big O, demostrando la importancia de seleccionar algoritmos eficientes cuando se trabaja con grandes volúmenes de información.